

Lineárna algebra pre neurónové siete - Ján Dobák

1.1 Základné skalárne operácie

Čo robí operácia $\cdot 2$?

- Násobí každý prvok tenzora (vektor/matica) číslom 2
- Príklad: $[1,2,3] \cdot 2 = [2,4,6]$ $[1,2,3] \cdot 2 = [2,4,6]$

Čo robí operácia $+ 1$?

- Pripočíta 1 ku každému prvku tenzora
- Príklad: $[2,4,6] + 1 = [3,5,7]$ $[2,4,6] + 1 = [3,5,7]$

Prečo je vektorizovaná verzia rýchlejšia?

- Využíva paralelizáciu – operácia beží naraz nad veľkými blokmi dát (CPU/GPU/TPU)
-

1.2 Transpozícia a tvar tenzora

Analýza troch pokusov v JAX:

- Pokus 1: matica tvaru $(3,2)(3,2)$ po transpozícii $A1.T$ dá tvar $(2,3)(2,3)$, čo je správne.
- Pokus 2: pri `jnp.transpose(A2, (1, 0))` sa pôvodný tvar $(2,3)(2,3)$ tiež zmení na $(3,2)(3,2)$

Prečo sa vektor nemení transpozíciou?

- Pokus 3: $A3$ je 1D vektor tvaru $(6,)(6,)$, transpozícia $A3.T$ nemení tvar, lebo transpozícia je definovaná pre aspoň 2D tenzory
 - Aby transpozícia fungovala, vektor treba najprv reshapenúť na maticu, `A3.reshape(2, 3)`, a potom transponovať
-

1.3 Broadcasting a výkon

Sú výsledky manuálnej a vektorizovanej implementácie rovnaké?

- Áno

Ktorá verzia je rýchlejšia?

```
vectorized_time = 0.067877 seconds
manual_time = 0.182944 seconds
```

- Vektorizovaná verzia je 2,70x rýchlejšia.

Analýza rôznych riešení broadcastingu:

- `result1 = A * v` správne – broadcasting rozšíri vektor
 - `result2 = A * v.reshape(1, 4)` je tiež správne len explicitne udáva tvar
 - `result3 = A * v.reshape(4, 1)` vedie k chybe, lebo osi nesedia s tvarom `(3,4)(3,4)`
-

1.4 PyTorch verzia tých istých úloh

- V PyTorch sa používajú typy `torch.tensor` / `torch.Tensor` a operácie `*`, `+`, `.T`, resp. `torch.transpose`
- Logika úloh 1.1–1.3 zostáva rovnaká: vytvorenie tenzora, manuálna slučka, vektorizovaná verzia a porovnanie výsledkov aj času

Výsledky a výkon:

- Výsledné hodnoty a tvary pre JAX aj PyTorch sú rovnaké, líšia sa len typom objektu (`jnp.ndarray` vs `torch.Tensor`)
 - Pri menších úlohách býva PyTorch mierne rýchlejší kvôli menšiemu JIT overheadu JAX, pri veľkých batchoch sa rozdiely znižujú
-

1.5 Porovnanie JAX vs PyTorch

Sú výsledky rovnaké?

- Áno, pre základné operácie (násobenie skalárom, transpozícia, broadcasting) sú hodnoty aj shapes v JAX a PyTorch zhodné
- Rozdiel je v zápise tvaru: JAX vracia napr. `(3, 4)` ako Python tuple, kým PyTorch používa `torch.Size([3, 4])`

Hlavné rozdiely medzi frameworkmi:

- JAX používa funkcionálny prístup, immutable štruktúry a JIT kompiláciu cez XLA, čo je výhodné pre numerické výpočty a výskum
- PyTorch stavia na dynamickom výpočtovom grafe, umožňuje in-place operácie a má širší ekosystém pre deep learning

2.1 Implementácia dot product

Vysvetlite rozdiel medzi element-wise a maticovým násobením. Ktorý spôsob je jednoduchší pre pochopenie? Ktorý spôsob sa používa v neurónových sieťach?

- Element-wise násobenie - vynásobí prvky na rovnakých pozíciách, výsledkom je tenzor rovnakého tvaru: $a * b = [a_1b_1, a_2b_2, a_3b_3]$
 - Maticové násobenie (dot product) - Vykoná súčet súčinov prvkov: $a \cdot b = a_1b_1 + a_2b_2 + a_3b_3$ výsledkom je skalár
 - Jednoduchšie na pochopenie - Element-wise spôsob je intuitívnejší, pretože rozkladá výpočet na menšie kroky (najprv násobenie po prvkoch, potom sčítanie).
 - Použitie v neurónových sieťach: maticové násobenie
 - Optimalizované pre GPU/TPU
 - Batch processing
-

2.2 Prečo musí byť počet stĺpcov $W ==$ dĺžka x ? Aký je výsledný shape? Prečo?

- Počet stĺpcov matice W sa musí rovnať dĺžke vektora x , pretože maticové násobenie funguje ako súčet súčinov zodpovedajúcich prvkov z riadku a vektora
 - Ak má matica W m riadkov a n stĺpcov, výsledkom násobenia s vektorom x (dĺžky n) je vektor s m prvkami
 - Každý riadok matice sa vynásobí po zložkách s vektorom x a dá jedno číslo. Keďže máme m riadkov, dostaneme m čísel – teda výsledný vektor dĺžky m .
-

2.3 Prečo sa ReLU používa namiesto tanh v moderných sieťach?

- Rýchlosť výpočtu - ReLU je jednoduchšia funkcia. Stačí porovnať hodnotu s nulou a prípadne ju vynulovať, zatiaľ čo tanh vyžaduje zložitejšie exponenciálne operácie
- Riešenie miznúceho gradientu - pre kladné vstupy má ReLU gradient presne 1, takže gradient prechádza sieťou bez zoslabenia. Tanh naopak pri veľkých hodnotách má gradient blízky 0, čo výrazne spomaľuje učenie
- Riedke reprezentácie - ReLU vytvára veľa nulových aktivácií, čo znamená efektívnejšiu reprezentáciu dát

- Rýchlejší tréning - v praxi vedú všetky tieto vlastnosti k výrazne rýchlejšiemu trénovaniu hlbokých neurónových sietí
 - Hlavný problém tanh - gradient funkcie tanh sa pri extrémnych hodnotách blíži k nule, čo spôsobuje miznutie gradientu a výrazne spomaľuje alebo úplne zastavuje učenie v hlbokých vrstvách siete
-

2.5 Porovnanie výsledkov a výkonu

Sú výsledky numericky rovnaké?

- Áno

Ktorý framework je rýchlejší?

- PyTorch bol výrazne rýchlejší vo všetkých troch úlohách:
 - Úloha 2.1: PyTorch 0.000355s vs JAX 0.046013s
 - PyTorch je 129,61x rýchlejší
 - Úloha 2.2: PyTorch 0.000335s vs JAX 0.089285s
 - PyTorch je 266,52x rýchlejší
 - Úloha 2.3: PyTorch 0.000216s vs JAX 0.033123s
 - PyTorch je 153,35x rýchlejší

Prečo je PyTorch rýchlejší pri malých úlohách?

- JAX overhead - JAX používa JIT (Just-In-Time) kompiláciu cez XLA, ktorá má inicializačný overhead pri prvom spustení
- PyTorch používa dynamický výpočtový graf bez JIT kompilácie, preto je pri malých operáciách okamžitý

Kedy by JAX mohol byť výhodnejší?

- Pri veľkých batchoch a zložitejších výpočtoch sa JIT kompilácia vyplatí
 - Po prvej kompilácii sa funkcia cachuje a následné volania sú rýchlejšie
-

ULOHA 2.1 - DOT PRODUCT:

JAX vysledok: 32.0

PyTorch vysledok: 32.0

Zhoda: ANO

Cas JAX: 0.046013s

Cas PyTorch: 0.000355s

Rychlejsi: PyTorch

ULOHA 2.2 - MATICOVE NASOBENIE:

JAX vysledok: [2.5 5.5]

PyTorch vysledok: [2.5 5.5]

Zhoda: ANO

Cas JAX: 0.089285s

Cas PyTorch: 0.000335s

Rychlejsi: PyTorch

ULOHA 2.3 - FORWARD PASS (ReLU):

JAX vysledok: [2.6 5.3]

PyTorch vysledok: [2.6 5.3]

Zhoda: ANO

Cas JAX: 0.033123s

Cas PyTorch: 0.000216s

Rychlejsi: PyTorch

3.1 Implementácia normy "ručne" vs vektorizovane

Sú výsledky rovnaké?

- Áno

Koľkokrát je vektorizovaná verzia rýchlejšia?

- Vektorizovaná verzia je v Pythone typicky o rády rýchlejšia (100x+), pretože nahrádza pomalú Python slučku optimalizovaným C/CUDA kódom.

- V JAX teste nižšie: 8,53x rýchlejšia.
- V PyTorch teste nižšie: 39,56x rýchlejšia.

Prečo je vektorizovaná verzia rýchlejšia?

- Využíva paralelizáciu (SIMD na CPU, masívna paralelizácia na GPU), menej overheadu interpretéra a efektívnejšiu prácu s pamäťou.

3.2 Batch processing

Prečo musíme použiť $W.T$ namiesto W pri batch processingu?

- X má tvar (batch_size, features), W má tvar (neurons, features).
- Pre maticové násobenie potrebujeme, aby vnútorné dimenzie sedeli: (batch, features) @ (features, neurons). Preto musíme W transponovať na (features, neurons).

Koľkokrát je vektorizovaná verzia rýchlejšia (podľa poznámok)?

- Vektorizovaná verzia je 2.58x rýchlejšia
-

3.4 Vektorizácia v PyTorch - flatten vs view

Aký je rozdiel medzi `torch.flatten()` a `.view()`?

- `torch.flatten()` je funkcia, ktorá vráti kópiu dát (ak treba) a umožňuje ľahko špecifikovať rozsah dimenzií na sploštenie (napr. `start_dim=1` pre zachovanie batch dimenzie).
 - `.view()` len mení pohľad na dáta (metadata o shape), nemení dáta v pamäti. Vyžaduje, aby tenzor bol "contiguous" (súvislý v pamäti). Ak nie je, vyhodí chybu (treba použiť `.contiguous().view()` alebo `.reshape()`).
-

3.5 Analýza viacvrstvovej siete

Koľko neurónov je aktívnych ($\text{ReLU} > 0$)?

- V skrytej vrstve 1: 8 z 20 (40.0%)
- V skrytej vrstve 2: 6 z 15 (40.0%)
- Množstvo aktívnych neurónov závisí od vstupných dát a náhodnej inicializácie váh.

Čo to znamená, ak je neurón "vypnutý"?

- Jeho výstup je 0. Pre daný vstup tento neurón neposiela ďalej žiadnu informáciu. V kontexte učenia (backprop) cez neho nepreteká gradient, takže sa v danom kroku neučí.

Čo znamená vysoká "energia" (súčet abs hodnôt váh)?

- Vysoká energia (napr. $V_2=251.47$) signalizuje veľké váhy. To môže naznačovať overfitting, kedy sieť reaguje príliš citlivo na vstupy.

Ako znížiť energiu?

- Regularizáciou (napr. L1 alebo L2), ktorá pridá penalizáciu za veľkosť váh do stratovej funkcie.

3.7 Porovnanie JAX vs PyTorch

- Výsledky sú numericky zhodné.
- V tomto teste bol PyTorch 129,06x rýchlejší (napr. Batch Forward: 0.0006s vs JAX 0.15s).
- Dôvodom je JIT kompilácia v JAXe (Just-In-Time), ktorá pri prvom spustení funkcie analyzuje a kompiluje kód, čo trvá dlhšie. Pri následných volaniach by bol JAX porovnateľne rýchly alebo rýchlejší, ale pre jednorazové malé skripty je PyTorch svižnejší (eager execution).

=== LINEAR ALGEBRA 2 - CAST 3 REPORT ===

--- Uloha 3.1 (JAX Norma) ---

Cas Manual: 0.402985s

Cas Vektorizovane: 0.047268s

Vysledky zhodne: ANO

--- Uloha 3.4.1 (PyTorch Norma) ---

Cas Manual: 0.007635s

Cas Vektorizovane: 0.000193s

Vysledky zhodne: ANO

--- Uloha 3.2 (JAX Batch Forward) ---

Cas Manual: 0.106085s

Cas Vektorizovane: 0.056719s

Vysledky zhodne: ANO

--- Uloha 3.4.2 (PyTorch Batch Forward) ---

Cas Manual: 0.002302s

Cas Vektorizovane: 0.001703s

Vysledky zhodne: ANO

--- Uloha 3.5 (JAX MLP Single - Manual vs Vectorized) ---

Cas Manual: 0.332981s

Cas Vektorizovane: 0.074307s

Vysledky zhodne: ANO

JAX ANALYZA (single):

Aktivne neurony V1: 8/20 (40.0%)

Aktivne neurony V2: 6/15 (40.0%)

Energia: V1=164.63, V2=251.47, V3=63.45

--- Uloha 3.6 (PyTorch MLP Single - Manual vs Vectorized) ---

Cas Manual: 0.003375s

Cas Vektorizovane: 0.000024s

Vysledky zhodne: ANO

=====

POROVNANIE FRAMEWORKOV (Vectorized Batch)

=====

Cas JAX: 0.135382s

Cas PyTorch: 0.001049s

Rychlejsi: PyTorch

Numericka zhoda: ANO

=====